



소프트웨어 버전 관리 & Tag & branch 관리 전략 – Git Flow

오늘의 학습내용

- 소프트웨어 버전관리
- Tag 생성 및 삭제
- branch 생성 전략 – Git Flow
- Git의 원리 – Working directory, Staging Area, Repository

◆ 소프트웨어 버전 관리

- ❖ 지속적인 소프트웨어 개발을 위해서는 효율적인 **소프트웨어 버전관리**가 필요합니다.
- ❖ 일반적으로는 **Major, Minor, Patch** 이렇게 3개의 구분 기호를 통해 버전을 관리합니다.

The screenshot displays the GitHub interface for the tensorflow/tensorflow repository. At the top, the repository name is shown with icons for Notifications, Star (154k), Fork (84.3k), and other repository statistics. Below this, a navigation bar includes links for Code, Issues (3.8k), Pull requests (200), Actions, Projects (1), Security (35), and Insights. The main content area is divided into 'Releases' and 'Tags' tabs, with 'Tags' currently selected. Under the 'Tags' tab, two tags are listed: v2.4.1 and v2.3.2. Each tag entry includes a clock icon indicating the release date, a commit hash, download links for zip and tar.gz files, a 'Notes' link, and a green 'Verified' badge.

Tag	Commit Hash	Release Date	Download Links	Notes	Verified
v2.4.1	85c8b2a	on 21 Jan	zip, tar.gz	Notes	Yes
v2.3.2	9edbe50	on 5 Jan	zip, tar.gz	Notes	Yes

◆ 소프트웨어 버전 관리 - Major

❖ 소프트웨어 버전 관리를 예를 들어 보면 다음과 같습니다.

v2.4.1



대규모 기능 추가 등 Major 업데이트가 이루어질 시에 변경

◆ 소프트웨어 버전 관리 - Minor

❖ 소프트웨어 버전 관리를 예를 들어 보면 다음과 같습니다.

v2.4.1



세부 기능 추가 등 Minor한 업데이트가 이루어질 시에 변경

◆ 소프트웨어 버전 관리 - Patch

❖ 소프트웨어 버전 관리를 예를 들어 보면 다음과 같습니다.

v2.4.1



버그 패치 등의 사소한 변경이 이루어질 시에 변경

◆ 소프트웨어 버전명을 기록하기 - tag

- ❖ 버전관리를 위해서는 각 버전에 대한 기록이 필요합니다.
- ❖ 이때 현재 소스코드 상태에 대한 기록을 **git tag** 명령어를 이용해서 생성할 수 있습니다.

```
git tag -a v2.4.1
```

▲ 생성된 tag 확인하기 - git tag -l

❖ 생성된 tag를 확인하기 위해서는 git tag -l 명령어를 사용하면 됩니다.

```
git tag -l
```

▲ 생성된 tag 원격 저장소에 반영하기

❖ 생성된 tag를 원격저장소에 반영하기 위해서는 tag명을 push해줍니다.

```
git push origin v0.0.1
```

◆ Git tag 실습 - tag 생성

❖ Git tag 명령어를 이용해서 tag 생성을 실습해봅시다.

◆ Git tag 삭제

❖ 생성한 tag를 삭제하기 위해서는 다음과 같은 명령어를 입력합니다.

```
git tag -d v0.0.1
```

◆ 원격저장소의 tag 삭제

❖ 원격 저장소에 올라간 tag를 삭제하기 위해서는 tag명 앞에 :를 붙입니다.

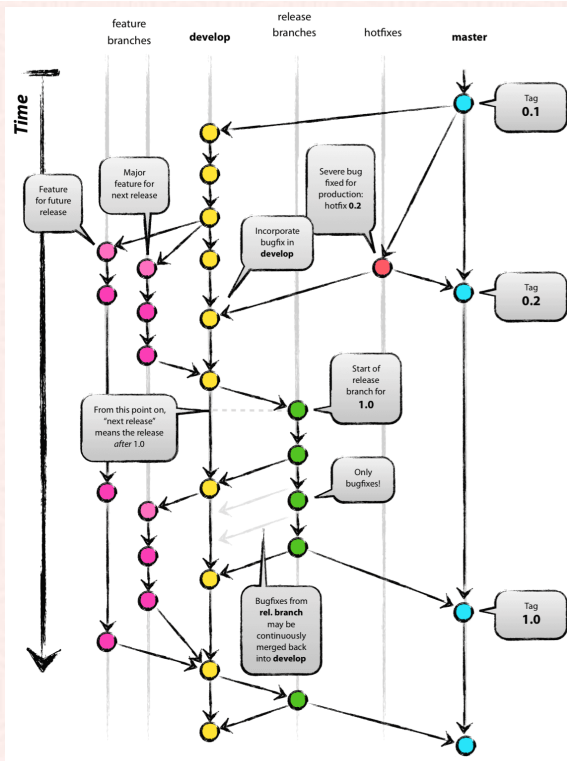
```
git push origin :v0.0.1
```

◆ Git tag 실습 - tag 삭제

❖ Git tag -d 명령어를 이용해서 tag 삭제를 실습해봅시다.

Branch 생성 전략 - Git Flow

- ❖ 효율적인 협업과 소프트웨어 개발을 위한 다양한 Branch 생성 전략들이 있습니다.
- ❖ 이 중에서 가장 유명한 Branch 생성 전략인 Vincent Driessen이 제안한 Git Flow 전략에 대해 살펴봅시다.



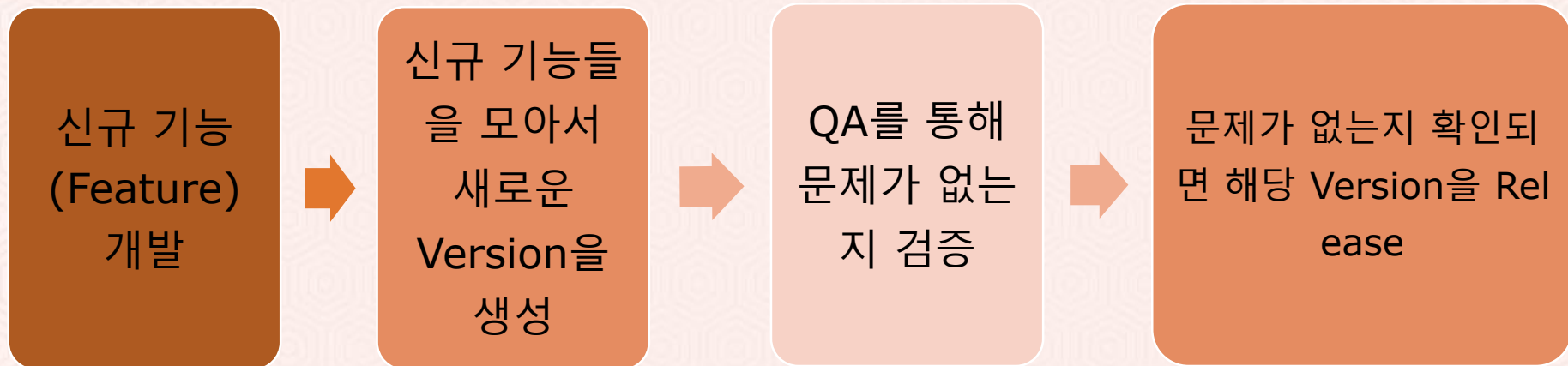
◆ Branch 생성 전략 - Git Flow

❖ Git Flow는 다음과 같이 5개의 branch를 이용해서 개발을 진행합니다.

1. **master** : 제품으로 출시될 수 있는 브랜치
2. **develop** : 다음 출시 버전을 개발하는 브랜치
3. **feature** : 기능을 개발하는 브랜치
4. **release** : 이번 출시 버전을 준비하는 브랜치
5. **hotfix** : 출시 버전에서 발생한 버그를 수정하는 브랜치

◆ 소프트웨어 출시 과정

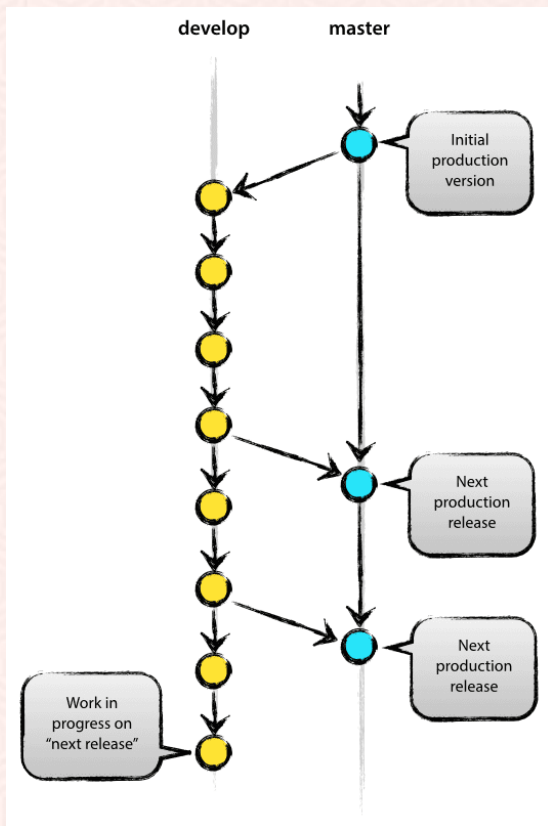
❖ 일반적으로 소프트웨어의 다음과 같은 과정을 거쳐서 출시합니다.



◆ Branch 생성 전략 - Git Flow

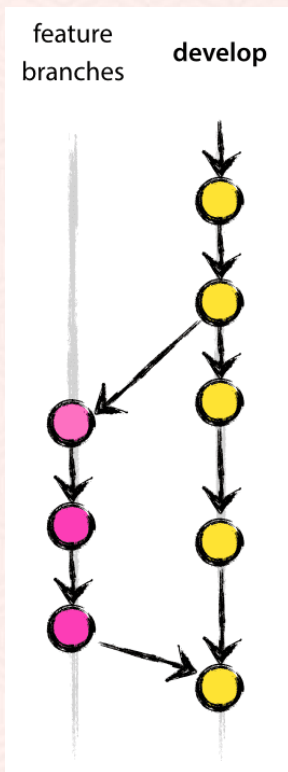
1. 일단 **master** 브랜치에서 시작을 합니다.
2. 동일한 브랜치를 **develop**에도 생성을 합니다. 개발자들은 이 develop 브랜치에서 개발을 진행합니다.
3. 개발을 진행하다가 회원가입, 장바구니 등의 기능 구현이 필요할 경우 A개발자는 develop 브랜치에서 **feature** 브랜치를 하나 생성해서 회원가입 기능을 구현하고 B개발자도 develop 브랜치에서 **feature** 브랜치를 하나 생성해서 장바구니 기능을 구현합니다.
4. 완료된 feature 브랜치는 검토를 거쳐 다시 develop 브랜치에 합칩니다.(Merge)
5. 이제 모든 기능이 완료되면 develop 브랜치를 **release** 브랜치로 만듭니다. 그리고 QA(품질검사)를 하면서 보완점을 보완하고 버그를 픽스합니다.
6. 모든 것이 완료되면 이제 release 브랜치를 master 브랜치와 develop 브랜치로 보냅니다. master 브랜치에서 버전추가를 위해 **tag**를 하나 생성하고 배포를 합니다.
7. 배포를 했는데 미처 발견하지 못한 버그가 있을 경우 **hotfixes** 브랜치를 만들어 긴급 수정 후 태그를 생성하고 바로 수정 배포를 합니다.

◆ Master와 develop branch



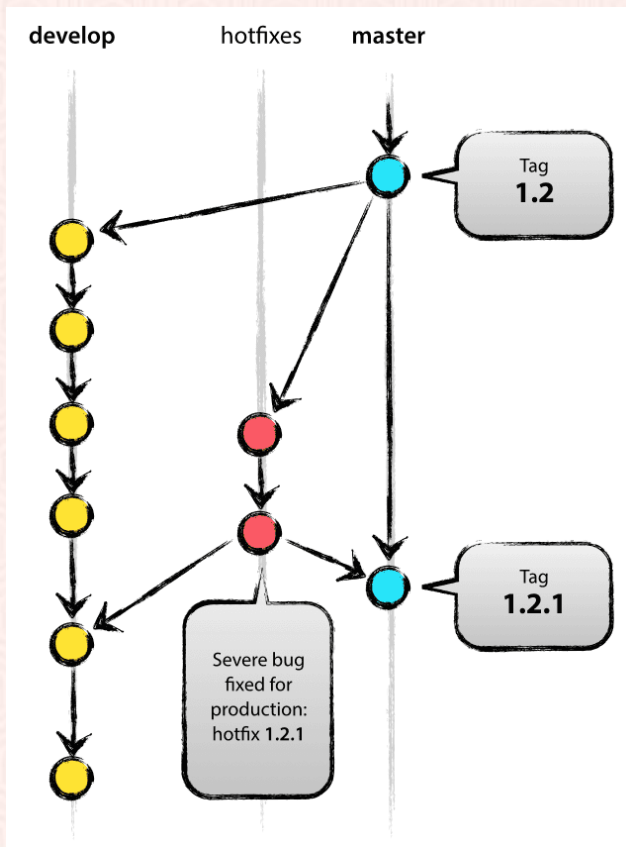
- ❖ **master** : 제품으로 출시될 수 있는 브랜치
- ❖ **develop** : 다음 출시 버전을 개발하는 브랜치
- ❖ master branch로부터 develop branch를 생성합니다.

◆ Develop branch와 feature branch



- ❖ **develop** : 다음 출시 버전을 개발하는 브랜치
- ❖ **feature** : 기능을 개발하는 브랜치
- ❖ develop branch로부터 feature branch를 생성합니다.

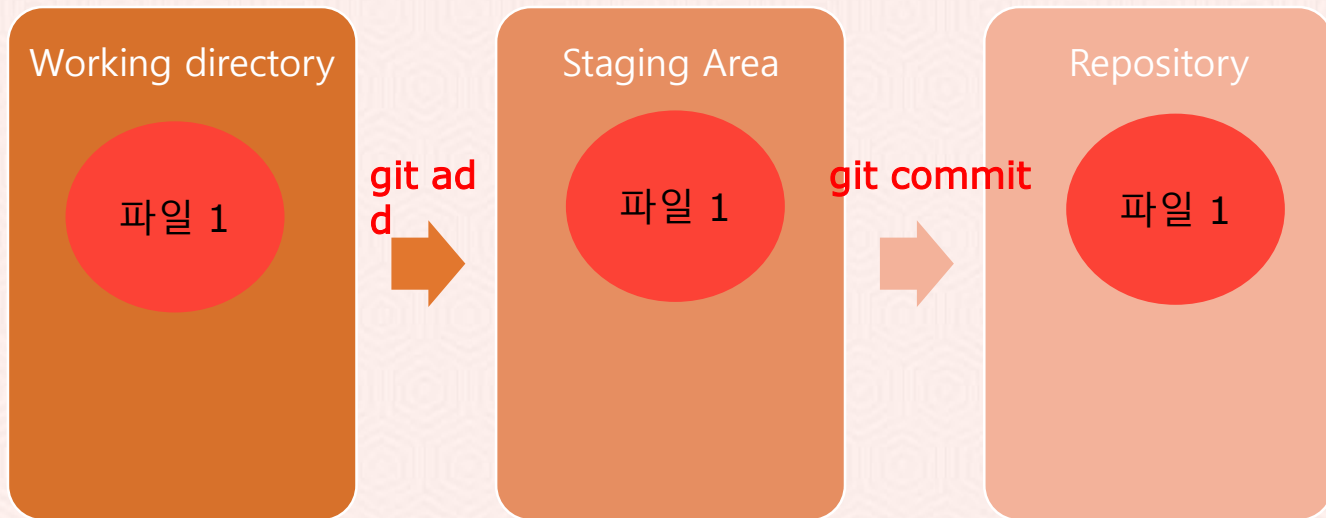
◆ hotfixes branch



- ❖ **hotfix** : 출시 버전에서 발생한 버그를 수정 하는 브랜치
- ❖ master branch에서 hotfixes branch를 생성합니다.

◆ Git의 원리 – Working directory, Staging Area, Repository

❖ Git에서 파일을 관리하는 Working directory, Staging Area, Repository의 개념을 살펴봅시다.



◆ 5강 학습내용 정리

❖ 5강 학습내용 정리

1. 소프트웨어 버전 관리 방법에 대해 살펴보았습니다.
2. git tag 명령어를 이용해서 tag를 생성하는 방법을 살펴보았습니다.
3. git tag 명령어를 이용해서 tag를 삭제하는 방법을 살펴보았습니다.
4. 소프트웨어 출시 과정에 대해 살펴보았습니다.
5. branch 생성 전략인 Git Flow 전략에 대해 살펴보았습니다.
6. Git의 원리 - Working directory, Staging Area, Repository에 대해 살펴보았습니다.